



UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final

Manutenção De Software - 01 - 2026.1

Profa. Carla Ilane Moreira Bezerra

Autores

Kendriks da Paixão

Sumário

Sumário	1
1. Caracterização do Projeto	3
2. Métricas Antes da Refatoração	3
2.1 Radon Antes da Refatoração	3
2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração	3
2.1.2 Complexidade Ciclomática por Função Antes da Refatoração	4
2.1.3 Índice de Manutenibilidade Antes da Refatoração	5
2.1.4 Métricas brutas Antes da Refatoração	5
2.2 CodeCarbon Antes da Refatoração	5
2.3 Pylint Antes da Refatoração	6
2.3.1 Distribuição do Smells Antes da Refatoração	6
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração	6
2.3.3 Arquivos mais Críticos Antes da Refatoração	7
2.3.4 Score Projeto Antes da Refatoração	7
2.4 Pytest Antes da Refatoração	8
2.4.1 Cobertura de testes Antes da Refatoração	8
2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração	10
3. Processo de Refatoração	10
3.1 too-many-statements	10
3.1.1 Ocorrência Número 1	11
3.1.2 Ocorrência Número 2	12
3.1.3 Ocorrência Número 3	13
3.1.4 Ocorrência Número 4	15
3.1.5 Ocorrência Número 5	16
3.2 too-many-instance-attributes	18
3.2.1 Ocorrência Número 1	18
3.2.2 Ocorrência Número 2	19
3.2.3 Ocorrência Número 3	21
3.2.4 Ocorrência Número 4	23
3.2.5 Ocorrência Número 5	25
3.2.6 Ocorrência Número 6	26
3.2.7 Ocorrência Número 7	28
3.3 too-many-branches	29
3.3.1 Ocorrência Número 1	29
3.3.2 Ocorrência Número 2	31
3.3.3 Ocorrência Número 3	33
3.3.4 Ocorrência Número 4	35
3.3.5 Ocorrência Número 5	36
3.3.6 Ocorrência Número 6	38

3.3.7 Ocorrência Número 7	40
4. Métricas Após a Refatoração	43
4.1 Radon Após a Refatoração	43
4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração	43
4.1.2 Complexidade Ciclomática por Função Após a Refatoração	43
4.1.3 Índice de Manutenibilidade Após a Refatoração	44
4.1.4 Métricas Brutas Após a Refatoração	45
4.2 CodeCarbon Após a Refatoração	45
4.3 Pylint Após a Refatoração	45
4.3.1 Distribuição do Smells Após a Refatoração	45
4.3.2 Distribuição dos Problemas de Código Após a Refatoração	46
4.3.3 Arquivos mais Críticos Após a Refatoração	46
4.3.4 Score Projeto Após a Refatoração	47
4.4 Pytest Após a Refatoração	47
4.4.1 Cobertura de testes Após a Refatoração	47
4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração	50
5. Comparação dos Resultados	50
6. Percepções sobre a Prática	50
7. Links	51

1. Caracterização do Projeto

Nome Projeto: Beets

O Beets é um sistema de gerenciamento de biblioteca musical voltado para entusiastas de música que desejam organizar e manter suas coleções de forma eficiente. Sua principal função é catalogar músicas e aprimorar automaticamente seus metadados, facilitando a organização e o acesso ao acervo.

Por meio de uma arquitetura baseada em plugins, o Beets oferece recursos avançados, como obtenção de capas de álbuns, letras, gêneros, tempos, níveis de ReplayGain e impressões digitais acústicas. Ele também integra informações de bases de dados especializadas, como MusicBrainz, Discogs e Beatport.

Além da organização, o Beets permite converter arquivos de áudio para diferentes formatos, identificar músicas duplicadas ou ausentes, limpar metadados incorretos, gerenciar capas de álbuns e analisar informações musicais pela linha de comando. Também oferece acesso à biblioteca por navegador web com suporte a áudio HTML5 e compatibilidade com players que utilizam o protocolo MPD.

Como é extensível, usuários podem desenvolver seus próprios plugins para adicionar funcionalidades específicas, tornando o Beets uma solução altamente personalizável para gerenciamento de coleções musicais.

2. Métricas Antes da Refatoração

2.1 Radon Antes da Refatoração

2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração

obs: usei csv e não peguei o último da planilha na tabela porque estava com valores vazios.

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
beets/ui/commands/update.py	2	18.50	34.0	37.0	E	update_items
beets/ui/commands/import_/session.py	11	9.45	32.0	104.0	E	choose_candidate
beets/util/hidden.py	1	9.00	9.0	9.0	B	is_hidden
beets/util/extension.py	2	9.00	14.0	18.0	C	fix_extension
beets/util/config.py	3	8.33	17.0	25.0	C	sanitize_pairs

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
beets/context.py	3	1.0	1.0	3.0	A	get_music_dir
beets/library/exceptions.py	4	1.0	1.0	4.0	A	__init__
beets/library/__init__.py	1	1.0	1.0	1.0	A	__getattr__
beets/ui/commands/__init__.py	1	1.0	1.0	1.0	A	__getattr__
beets/test/_common.py	16	1.19	2.0	19.0	A	item

2.1.2 Complexidade Ciclomática por Função Antes da Refatoração

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
beets/ui/__init__.py	function	input_options	E	40
beets/ui/commands/update.py	function	update_items	E	34
beets/ui/commands/import_/session.py	function	choose_candidate	E	32
beets/autotag/distance.py	function	distance	D	30
beets/util/layout.py	function	split_into_lines	D	26

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
beets/context.py	function	get_music_dir	A	1
beets/logging.py	function	__init__	A	1
beets/metadata_plugins.py	method	data_source	A	1
beets/plugins.py	method	commands	A	1
beets/autotag/distance.py	function	get_track_length_grace	A	1

2.1.3 Índice de Manutenibilidade Antes da Refatoração

Tabela de piores métricas

arquivo	mi	rank_mi
beets/library/models.py	6.2092	C
beets/dbcore/db.py	12.5145	B
beets/importer/tasks.py	13.4389	B
beets/dbcore/query.py	24.2219	A
beets/util/__init__.py	26.3283	A

Tabela de melhores métricas

arquivo	mi	rank_mi
beets/importer/__init__.py	100.0	A
beets/context.py	100.0	A
beets/exceptions.py	100.0	A
beets/ui/commands/__init__.py	100.0	A
beets/ui/commands/version.py	100.0	A

2.1.4 Métricas brutas Antes da Refatoração

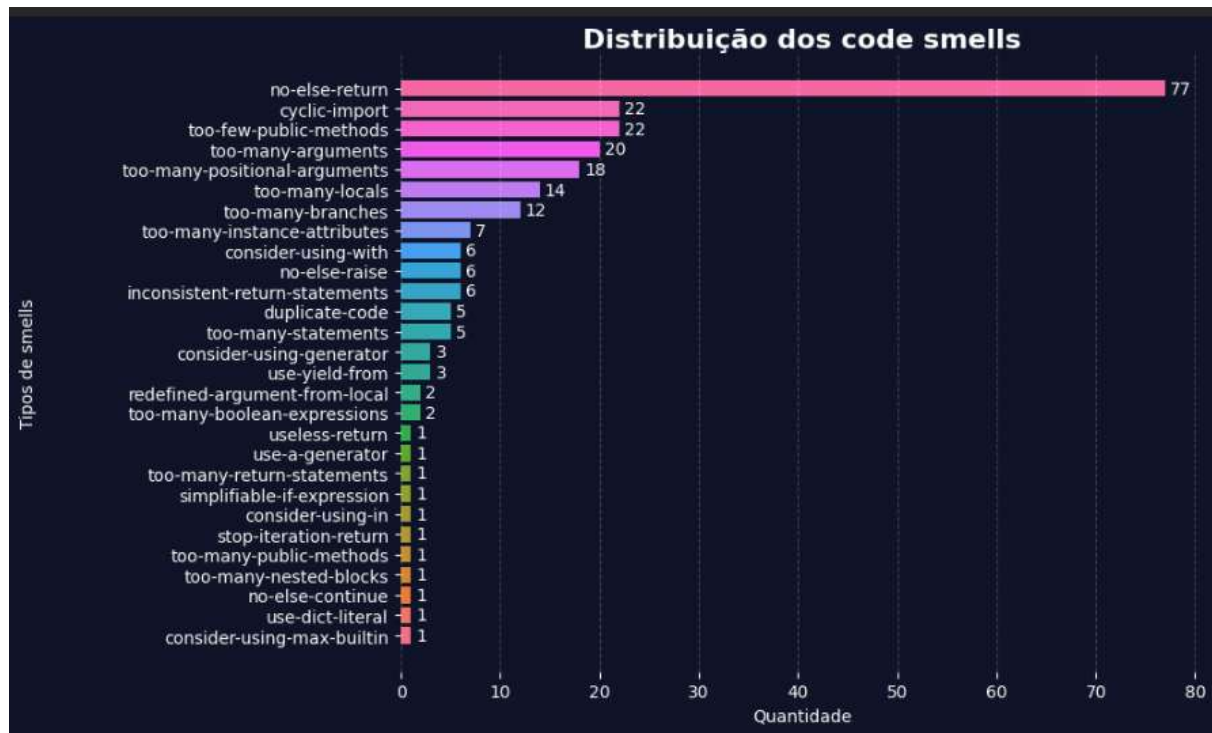
loc total	lloc total	sloc total	comments total	multi
21067	10492	12438	1949	2750

2.2 CodeCarbon Antes da Refatoração

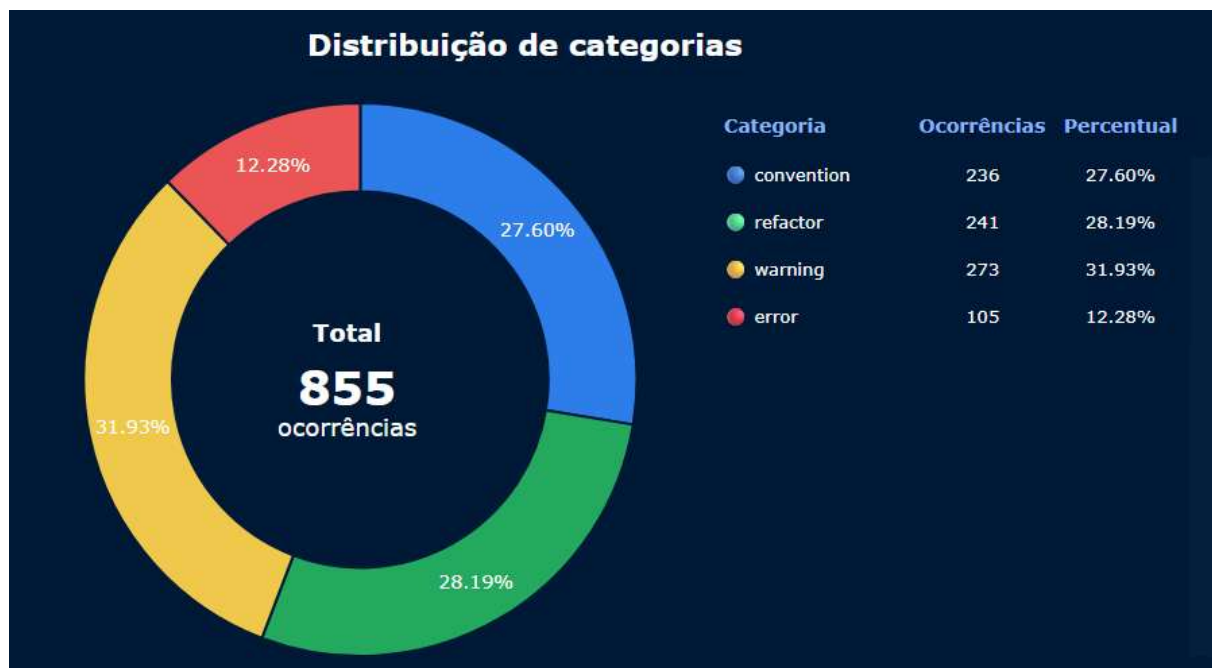
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
1.6196625	3.05E-07	1.89E-07	0	3.11E-06	3.11E-06

2.3 Pylint Antes da Refatoração

2.3.1 Distribuição do Smells Antes da Refatoração



2.3.2 Distribuição dos Problemas de Código Antes da Refatoração



2.3.3 Arquivos mais Críticos Antes da Refatoração



2.3.4 Score Projeto Antes da Refatoração



2.4 Pytest Antes da Refatoração

2.4.1 Cobertura de testes Antes da Refatoração

Code Coverage Report					
File	statements	missing	excluded	coverage	
beets/_init_.py	18	5	0		70%
beets/_main_.py	4	4	0		0%
beets/autotag/_init_.py	12	4	0		57%
beets/autotag/distance.py	261	29	5		86%
beets/autotag/hooks.py	217	1	2		99%
beets/autotag/match.py	213	16	9		90%
beets/context.py	13	0	0		100%
beets/dbcore/_init_.py	5	0	0		100%
beets/dbcore/db.py	626	25	8		94%
beets/dbcore/pathutils.py	29	1	0		95%
beets/dbcore/query.py	427	29	7		92%
beets/dbcore/queryparse.py	79	1	4		98%
beets/dbcore/sort.py	107	8	2		94%
beets/dbcore/types.py	237	5	8		97%
beets/exceptions.py	1	0	0		100%
beets/importer/_init_.py	3	0	0		100%
beets/importer/session.py	142	11	9		91%

beetsimporter/season.py	142	11	9		91%
beetsimporter/stages.py	169	13	4		89%
beetsimporter/state.py	57	5	2		89%
beetsimporter/tasks.py	553	28	6		93%
beetslibrary/_init_.py	9	0	0		100%
beetslibrary/exceptions.py	14	0	0		100%
beetslibrary/fields.py	2	0	0		100%
beetslibrary/library.py	88	5	4		94%
beetslibrary/migrations.py	114	1	3		97%
beetslibrary/models.py	647	60	6		88%
beetslibrary/queries.py	22	0	0		100%
beetslogging.py	62	1	12		96%
beetsmediafile.py	7	7	0		0%
beetsmetadata_plugins.py	164	23	9		82%
beetsplugins.py	242	19	17		89%
beetsui/_init_.py	417	53	2		86%
beetsui/commands/_init_.py	18	0	0		100%
beetsui/commands/completion.py	52	41	0		15%
beetsui/commands/config.py	43	6	0		83%
beetsui/commands/fields.py	20	1	0		91%
beetsui/commands/help.py	13	0	0		100%
beetsui/commands/import/_init_.py	107	57	0		40%
beetsui/commands/import/_display.py	201	27	5		82%
beetsui/commands/import/_session.py	228	74	0		63%
beetsui/commands/list.py	13	0	0		100%
beetsui/commands/modify.py	67	3	0		96%
beetsui/commands/move.py	74	15	2		75%
beetsui/commands/remove.py	36	0	0		100%
beetsui/commands/stats.py	35	5	0		81%
beetsui/commands/update.py	87	13	0		83%
beetsui/commands/utls.py	15	0	0		100%
beetsui/commands/version.py	13	1	0		87%
beetsui/commands/write.py	25	5	0		81%
beetsutil/_init_.py	496	94	5		80%
beetsutil/artresizer.py	358	150	5		54%
beetsutil/bluelet.py	333	251	0		19%
beetsutil/color.py	56	0	0		100%
beetsutil/config.py	42	0	2		100%
beetsutil/deprecation.py	34	2	2		93%
beetsutil/diff.py	47	0	4		100%
beetsutil/extension.py	62	28	0		41%
beetsutil/functiontemplate.py	272	24	0		90%
beetsutil/hidden.py	21	8	0		49%
beetsutil/id_extractors.py	14	3	0		81%
beetsutil/layout.py	152	18	3		85%
beetsutil/lyrics.py	91	2	2		97%
beetsutil/m3u.py	40	6	0		84%
beetsutil/pathformats.py	7	0	4		100%
beetsutil/pipeline.py	234	11	2		94%
beetsutil/units.py	30	0	0		100%
beetsplug/_util/_init_.py	2	0	0		100%
beetsplug/_util/start.py	94	30	0		65%
beetsplug/_util/musicbrainz.py	340	7	5		97%
beetsplug/_util/playcount.py	48	1	5		97%
beetsplug/_util/requests.py	60	0	6		98%
beetsplug/_util/shs.py	21	0	2		96%
beetsplug/accounts/brainz.py	95	35	0		38%
beetsplug/advancedrewrite.py	74	7	0		88%
beetsplug/albumtypes.py	26	2	2		91%
beetsplug/boothies.py	155	94	0		26%

beetsplugbareasc.py	40	8	0		74%
beetsplugbepart.py	239	139	15		39%
beetsplugbpd_init_.py	884	862	4		2%
beetsplugbpdqstplayer.py	156	169	0		3%
beetsplugbucket.py	117	0	0		100%
beetsplugchroma.py	267	125	5		49%
beetsplugconvert.py	317	73	5		74%
beetsplugdiscogs_init_.py	324	110	5		65%
beetsplugdiscogsstates.py	121	3	5		96%
beetsplugdiscogslyrics.py	42	0	2		100%
beetsplugduplicates.py	165	67	0		54%
beetsplugedit.py	168	16	0		88%
beetsplugembedart.py	125	27	0		77%
beetsplugembedupdate.py	75	30	0		54%
beetsplugexport.py	103	10	2		88%
beetsplugfetchart.py	733	137	5		77%
beetsplugfilter.py	34	3	0		85%
beetsplugfromfilename.py	66	2	0		95%
beetspluginitio.py	113	9	3		89%
beetsplugfuzzy.py	26	0	0		97%
beetsplughook.py	38	5	0		84%
beetsplughelo.py	36	19	0		44%
beetsplugimportadded.py	76	1	0		95%
beetsplugimportfeeds.py	79	11	0		80%
beetsplugimportsource.py	68	14	0		75%
beetspluginfo.py	121	17	0		84%
beetspluginline.py	68	9	0		86%
beetsplugipfs.py	206	160	0		20%
beetsplugkeyfinder.py	46	6	0		86%
beetspluglastgenre_init_.py	262	31	11		84%
beetspluglastgenreclient.py	41	18	7		51%
beetspluglastgenreutils.py	15	0	5		100%
beetspluglimit.py	43	4	0		86%
beetspluglinbrainz.py	229	85	3		62%
beetspluglyrics.py	516	93	11		78%
beetsplugmbcollection.py	103	4	6		93%
beetsplugmpseudo.py	156	15	5		86%
beetsplugmbsubmit.py	36	17	0		52%
beetsplugmysync.py	79	8	0		84%
beetsplugmetasync_init_.py	69	10	2		85%
beetsplugmetasyncamarok.py	39	20	0		39%
beetsplugmetasyncitunes.py	56	9	0		80%
beetsplugmissing.py	88	17	3		75%
beetsplugmpdata.py	192	78	0		54%
beetsplugmusicbrainz.py	320	9	5		96%
beetsplugnetwork.py	92	15	2		78%
beetsplugpermissions.py	53	31	0		33%
beetsplugplay.py	112	14	0		85%
beetsplugplaylist.py	109	8	3		91%
beetsplugplexupdate.py	47	11	0		74%
beetsplugrandom.py	49	4	4		91%
beetsplugreplace.py	73	33	2		54%
beetsplugreplaygen.py	676	539	9		16%
beetsplugrewrite.py	50	1	0		98%
beetsplugscrub.py	67	24	0		66%
beetsplugsmartplaylist.py	216	15	3		91%
beetsplugspotify.py	305	109	6		69%
beetsplugsubsonicupdate.py	63	13	0		75%
beetsplugsubstitute.py	13	1	0		88%
beetsplugthe.py	53	6	0		82%

beetsplugthumbs.py	145	34	1		77%
beetsplugtidal_init_.py	165	18	35		88%
beetsplugtidalapi.py	83	56	4		26%
beetsplugtidalsession.py	50	30	2		34%
beetsplugtidalcase.py	128	0	4		99%
beetsplugtypes.py	25	1	0		95%
beetsplugweb_init_.py	272	73	0		71%
beetsplugzero.py	90	4	0		95%
extrarelease.py	113	101	0		9%
extract_metrics_before_codecarbon.py	28	28	0		0%
extract_metrics_before_pylintr.py	58	58	0		0%
extract_metrics_before_pytest.py	32	32	0		0%
extract_metrics_before_radon.py	111	111	0		0%
extract_score_before_pylintr.py	13	13	0		0%
Total	19746	5228	359		71%
Total	39492	10456	718		74%

2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração

Quantidade de testes que passaram: 2341

Quantidade de testes que falharam: 10

3. Processo de Refatoração

Modelo utilizado: GPT-5.4 mini (plano free)

3.1 too-many-statements

Quantidade de ocorrências: 5

3.1.1 Ocorrência Número 1

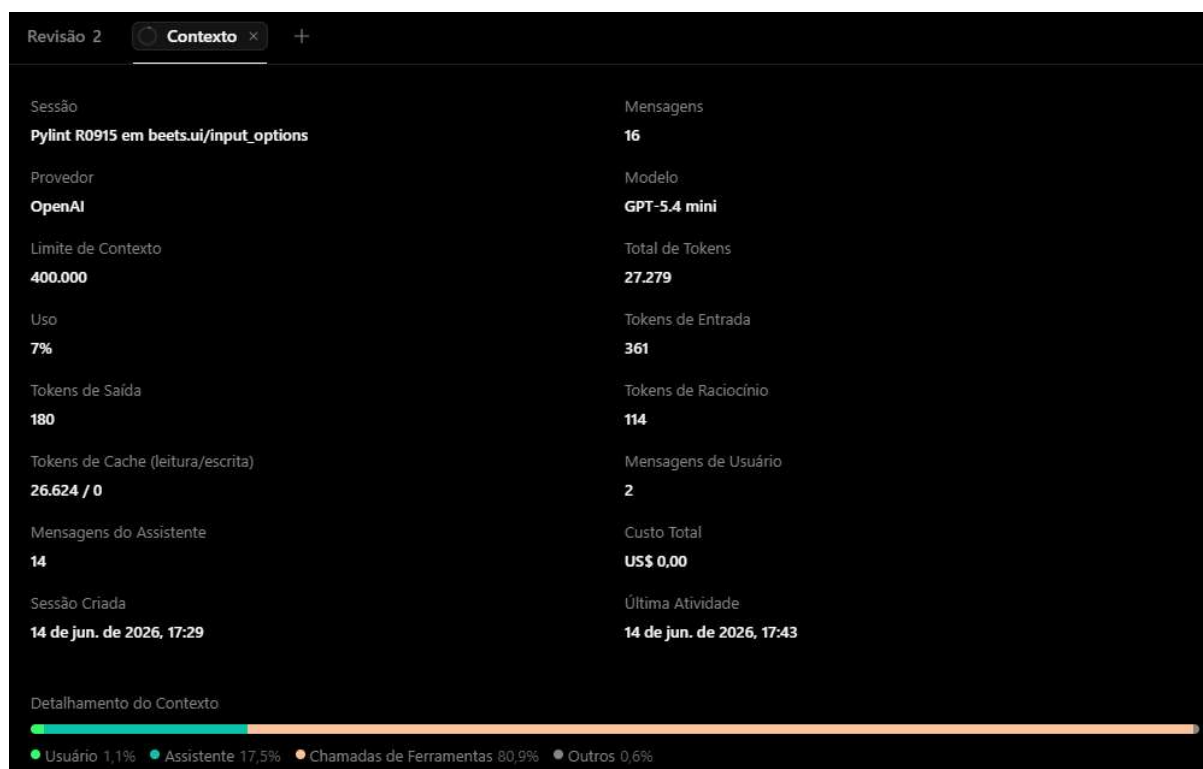
Tabela de identificação

module	obj	message
beets.ui	input_options	Too many statements (87/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolve corretamente a ocorrência do smell, pois a função está fazendo muitas responsabilidades(87 statements), e a divisão proposta separa partes naturais da lógica sem mudar o comportamento. Isso reduz o R0915, melhora a leitura e facilita a manutenção e testes.

Link da publicação OpenCode: <https://opncl.ai/share/EhAV2UuG>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função estava grande e difícil de manter, e a divisão em helpers deixou o código mais claro, testável e dentro do limite do Pylint sem mudar o comportamento. O custo foi só um leve aumento de fragmentação e a validação ter sido indireta, mas nada que pese aqui. No geral, o ganho em organização e legibilidade compensa bem o esforço.

3.1.2 Ocorrência Número 2

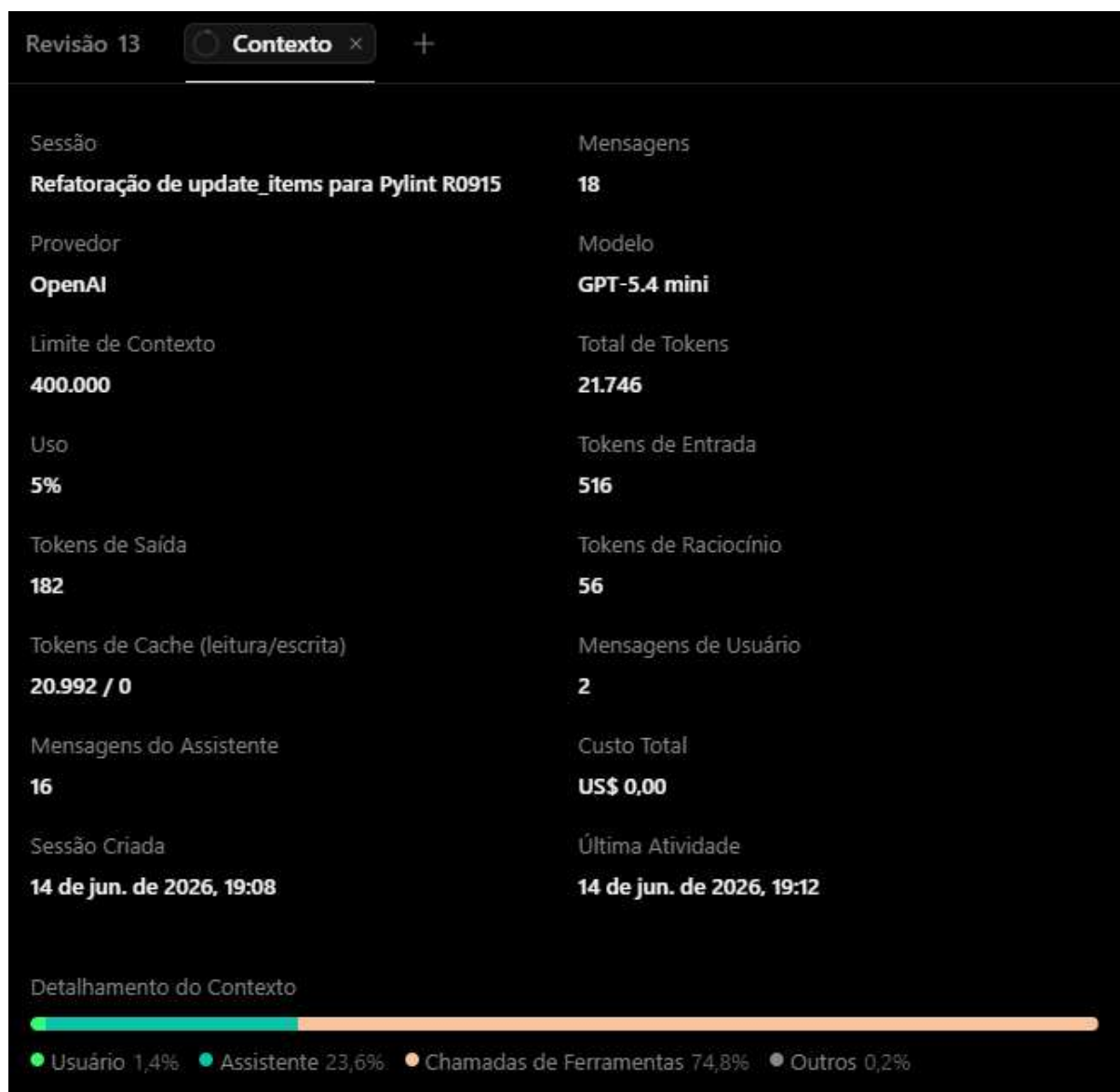
Tabela de identificação

module	obj	message
beets.ui.commands.update	update_items	Too many statements (62/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolve corretamente a ocorrência do smell, pois a função está fazendo muitas coisas ao mesmo tempo, o que aumenta a complexidade e risco de regressão. A divisão em helpers mantém o comportamento e melhora a legibilidade e manutenção sem quebrar a API.

Link da publicação OpenCode: <https://opncd.ai/share/al8l5VXx>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena. Mesmo com um custo de token alto, economizou tempo e reduziu o risco de erro em uma nova refatoração estrutural e repetitiva. No geral, o ganho em velocidade e segurança compensa.

3.1.3 Ocorrência Número 3

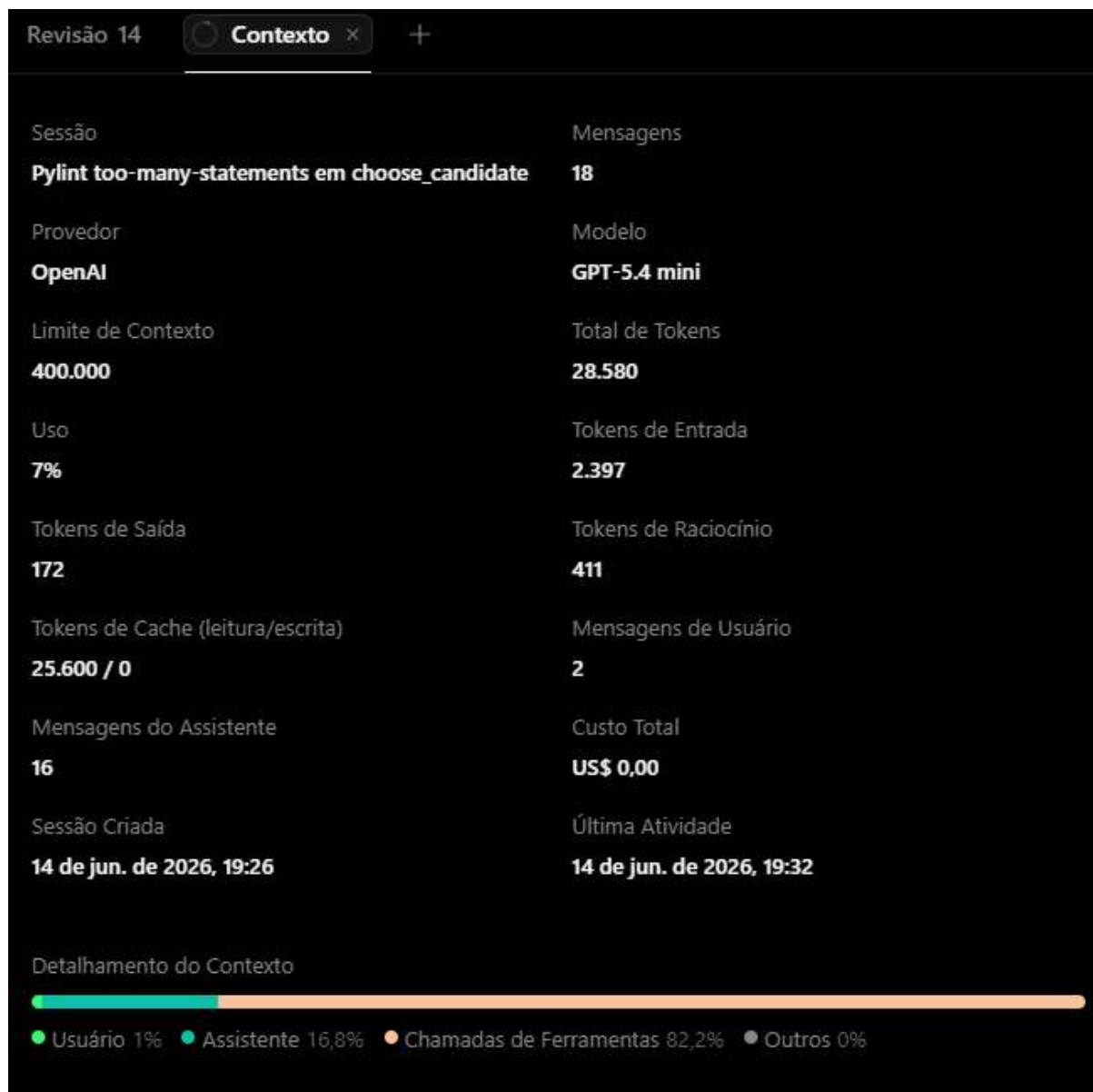
Tabela de identificação

module	obj	message
beets.ui.commands.import_session	choose_candidate	Too many statements (59/50)

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, pois o `choose_candidate` está com muitas responsabilidades misturadas e separar em helpers deixa o fluxo mais claro e reduz o R0915 sem mudar o comportamento.

Link da publicação OpenCode: <https://opncd.ai/share/kz8Kw1pl>

Print consumo de token:



Análise da refatoração: No geral, valeu a pena se pensar em relação a segurança e qualidade, porque ajudou a refatorar sem quebrar o comportamento. Porém, o custo de tempo e tokens foi muito alto considerando que é uma tarefa basicamente mecânica. No geral, compensa mais pelo ganho de segurança do que pela eficiência.

3.1.4 Ocorrência Número 4

Tabela de identificação

module	obj	message
beets.util.bluelet	run	Too many statements (88/50)

Seguiu o plano de refatoração:

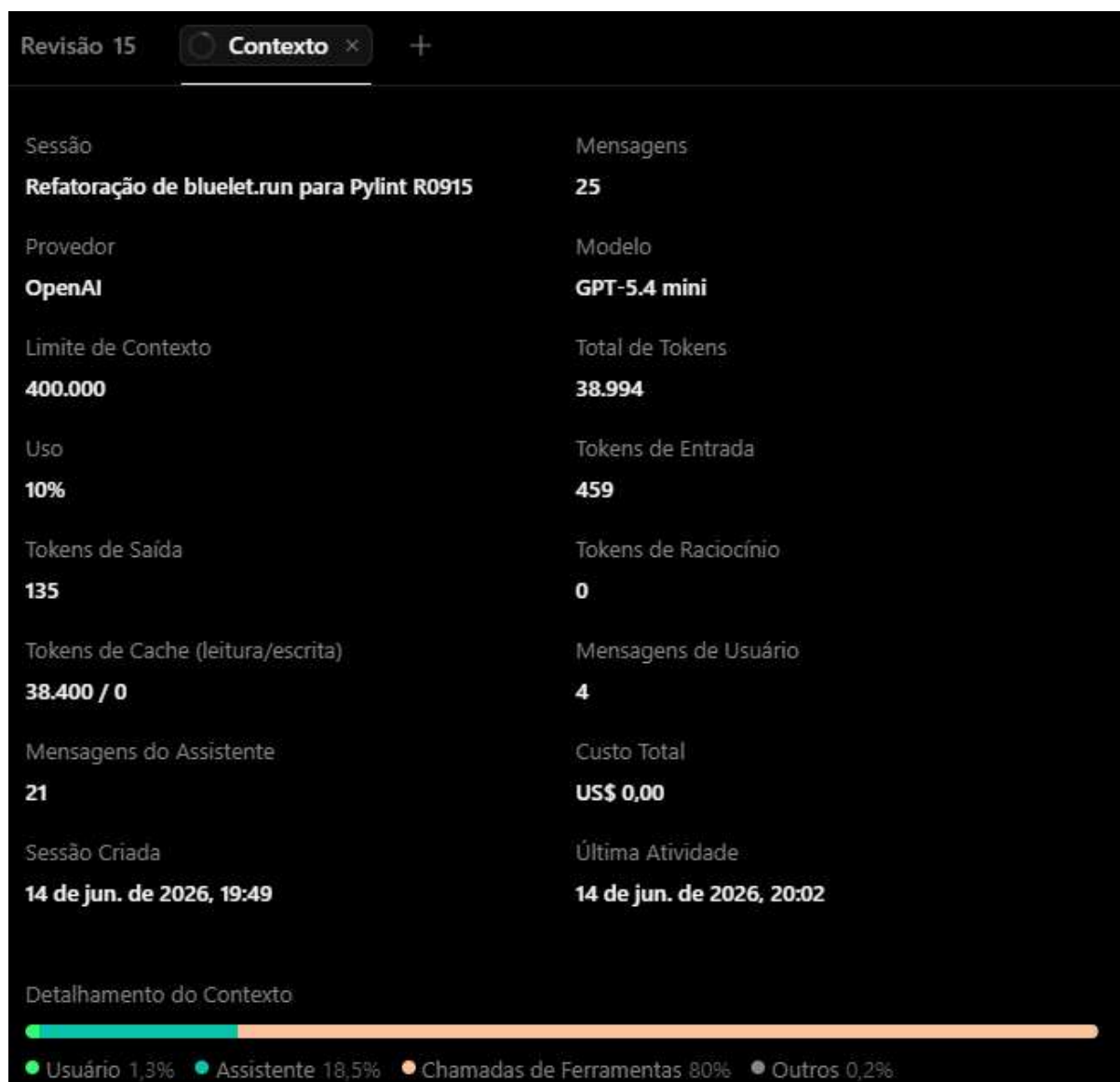
Sobre a primeira sugestão: Sim, run está grande e com várias responsabilidades, então extrair helpers reduz o R0915 e melhora a leitura sem mudar a API. Só exige um pouco de cuidado porque é um fluxo sensível de execução.

FOI NECESSÁRIO UM SEGUNDO PLANO PORQUE APÓS O BUILD, NÃO REFATOROU O CODE SMELL.

Sobre a segunda sugestão: Sim, o `_run_scheduler` está grande e com várias responsabilidades, então extrair os helpers reduz o R0915 e melhora a leitura sem mudar o comportamento.

Link da publicação OpenCode: <https://opncd.ai/share/uPRluDpT>

Print consumo de token:



Análise da refatoração: Considero que a refatoração foi parcialmente boa. Foi muito cara em relação a tokens e tempo para uma refatoração mecânica. Foi preciso de mais que uma interação com o chat para conseguir refatorar, de fato. No geral, compensa mais pelo cuidado e eficiência.

3.1.5 Ocorrência Número 5

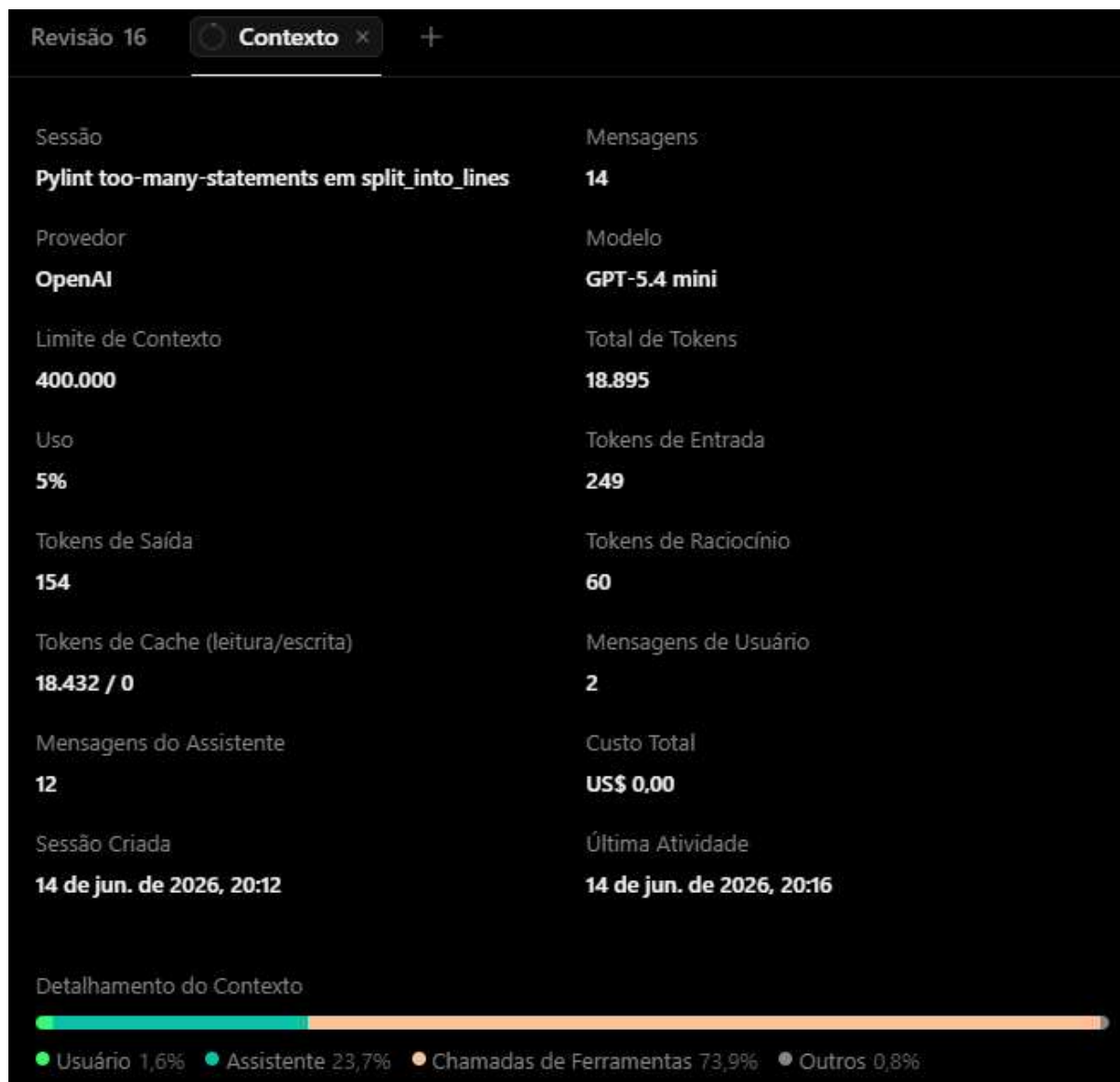
Tabela de identificação

module	obj	message
beets.util.layout	split_into_lines	Too many statements (66/50)

Seguiu o plano de refatoração: Sim, `split_into_lines` mistura detecção de ANSI, tokenização e wrapping, então extrair `_split_words_preserving_colors` e `_wrap_words` reduz o R0915 e separa responsabilidades sem alterar o comportamento.

Link da publicação OpenCode: <https://opncd.ai/share/zmCQHV3O>

Print consumo de token:



Análise da refatoração: Não foi uma refatoração muito eficiente em questão de tempo e tokens. Essa é uma refatoração um tanto quanto simples e previsível, então o custo para esse tipo de tarefa ficou muito alto. Assim como os outros, compensa mais pela segurança que pela eficiência.

3.2 too-many-instance-attributes

Quantidade de ocorrências: 7

3.2.1 Ocorrência Número 1

Tabela de identificação

module	obj	message
beets.plugins	BeetsPlugin	Too many instance attributes (8/7)

Seguiu o plano de refatoração:

Primeira sugestão: Eu não aprovei o primeiro plano porque a mudança não reduz complexidade real, só reduz contagem artificial de atributos.

Segunda sugestão: Eu aprovei, pois realmente estava separando as responsabilidades e reduzindo o acoplamento dentro do BeetsPlugin. O R0902 virou consequência, e não objetivo.

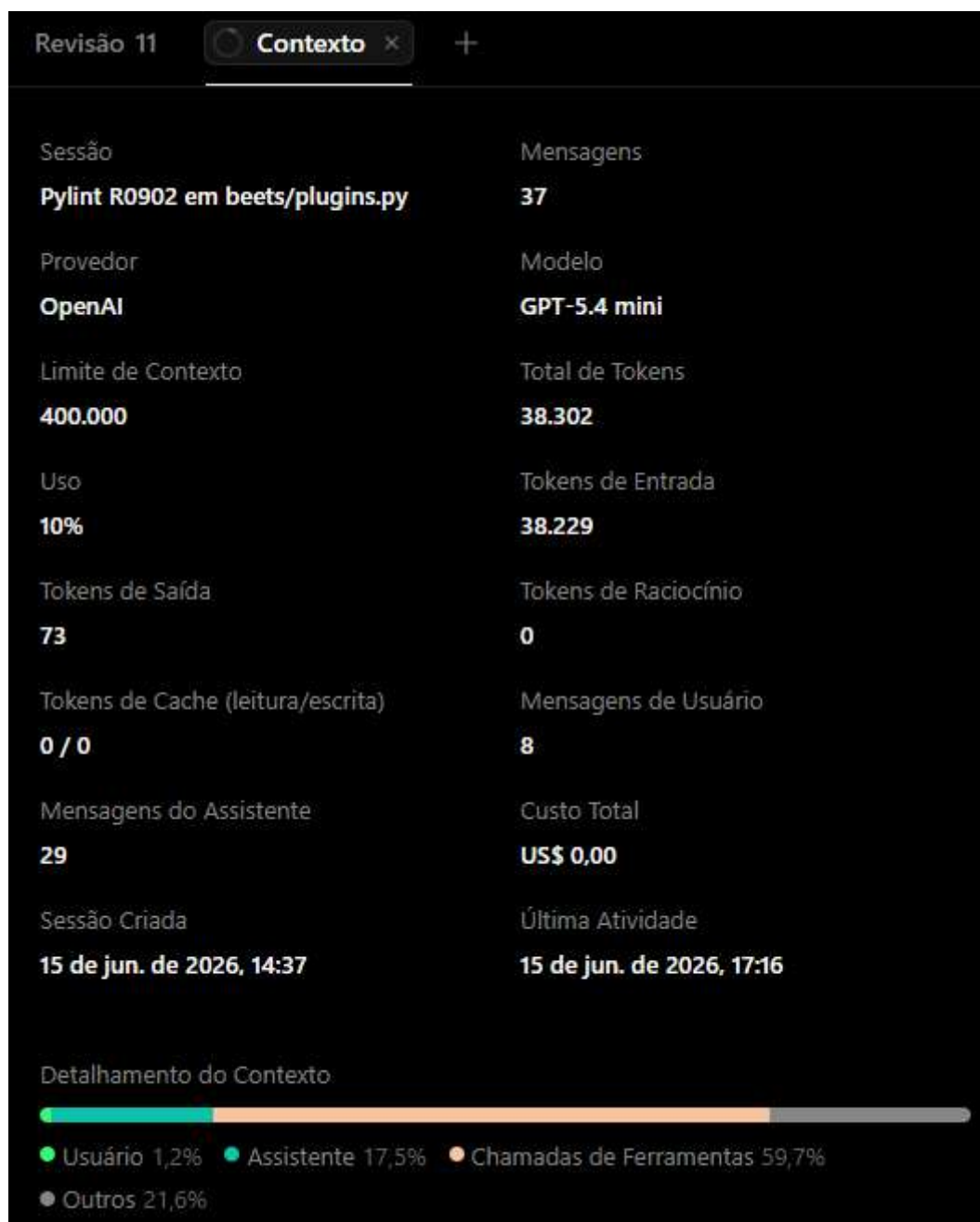
MESMO CUMPRINDO O PROPÓSITO, O CODE SMELL NÃO FOI REFATORADO.

Terceira sugestão: Eu não aprovei esse plano porque ela mistura uma boa refatoração com uma tentativa de “otimizar o Pylint”, adicionando uma complexidade desnecessária.

Quarta sugestão: Eu aprovei pois é a solução mais simples e honesta, não muda o design da classe nem adiciona complexidade só para satisfazer o lint.

Link da publicação OpenCode: <https://opncd.ai/share/6L7E2Wxg>

Print consumo de token:



Análise da refatoração: Eu considero que essa refatoração não valeu a pena. Ao todo, foram mais de 38 mil tokens para chegar à uma conclusão simples: o R0902 não justificava refatoração e uma supressão local era suficiente. O mesmo resultado poderia ter sido alcançado muito mais rápido.

3.2.2 Ocorrência Número 2

Tabela de identificação

module	obj	message
--------	-----	---------

beets.autotag.hooks	Info	Too many instance attributes (13/7)
---------------------	------	-------------------------------------

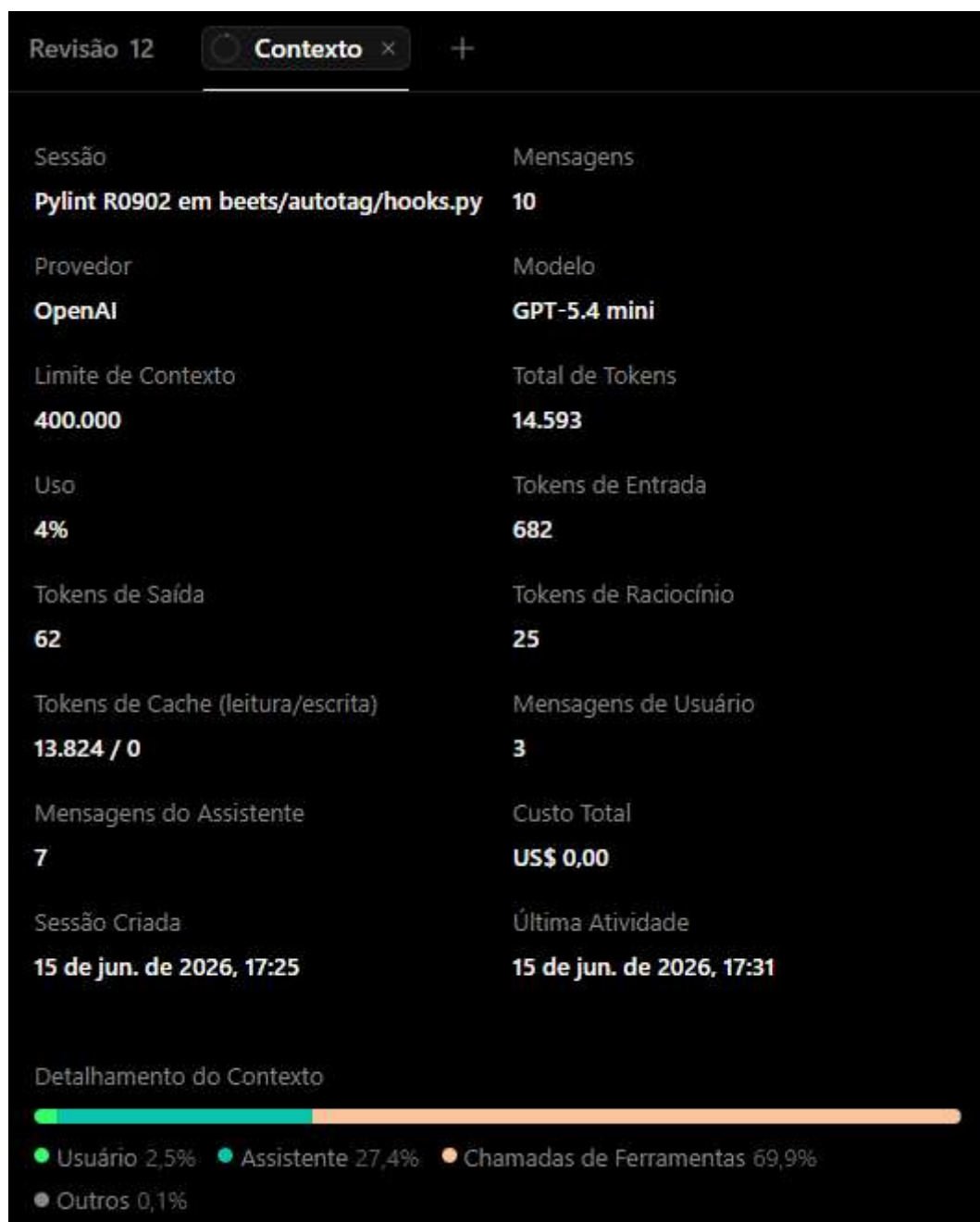
Seguiu o plano de refatoração:

Primeira sugestão: Eu não aprovei porque parece o mesmo problema de antes: mudar a implementação principalmente para satisfazer o Pylint.

Segunda sugestão: Eu aprovei porque Info é um contêiner de dados, então muitos atributos são esperados. Uma supressão local é mais simples e honesta do que refatorar só por causa do lint.

Link da publicação OpenCode: <https://opncd.ai/share/R97bFJDI>

Print consumo de token:



Análise da refatoração: Considero que valeu a pena pois, ao contrário da anterior, consegui identificar mais rápido o problema do plano e guiá-lo diretamente para uma solução mais assertiva.

3.2.3 Ocorrência Número 3

Tabela de identificação

module	obj	message
--------	-----	---------

beets.autotag.hooks	AlbumInfo	Too many instance attributes (28/7)
---------------------	-----------	-------------------------------------

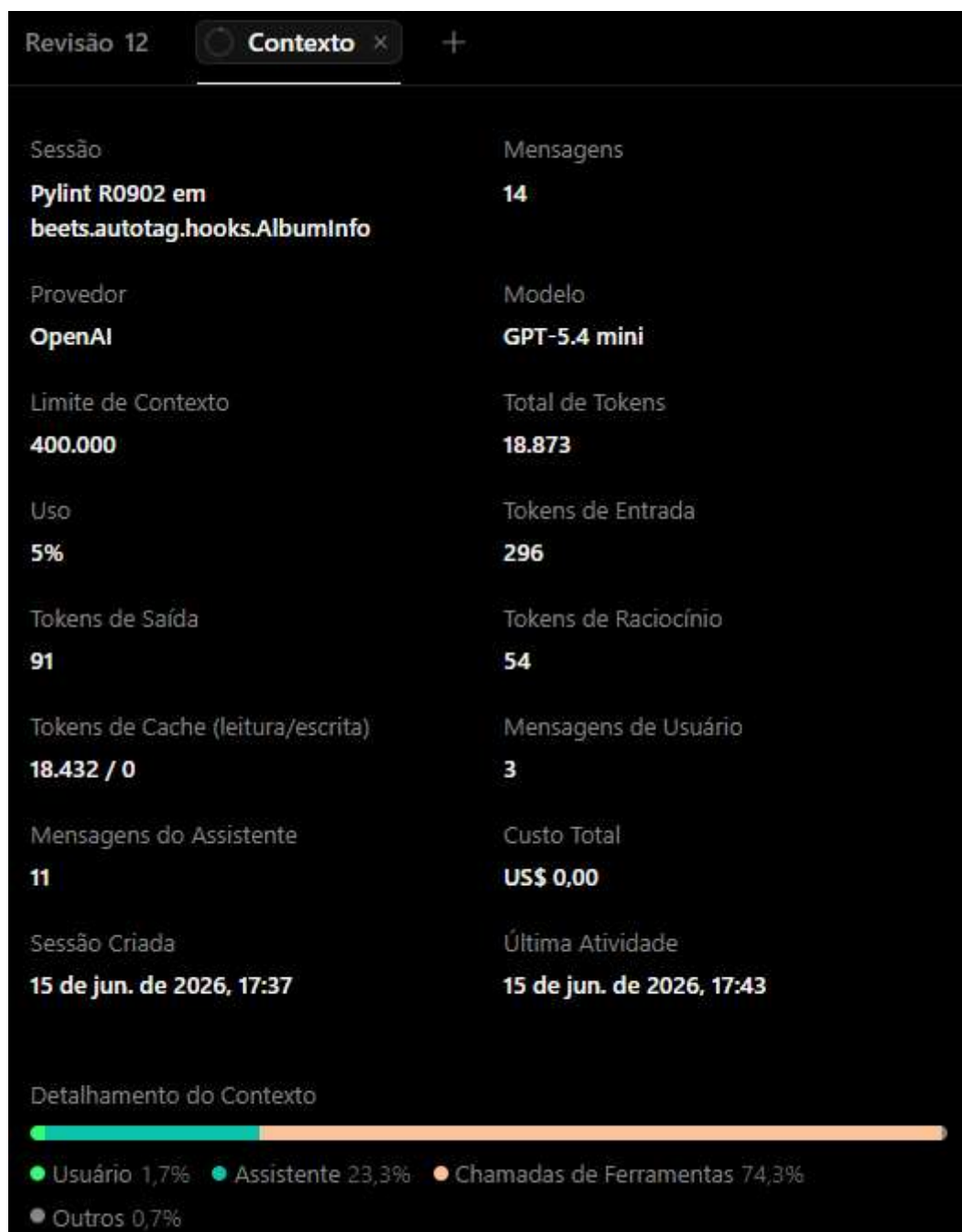
Seguiu o plano de refatoração:

Primeira sugestão: Eu não aprovei porque o problema é o mesmo: mudar a implementação principalmente para satisfazer o Pylint.

Segunda sugestão: Eu aprovei porque AlbumInfo é um contêiner de metadados, então é o esperado que tenha muitos atributos. Aqui o R0902 não indica um problema real de design, só um limite artificial. A supressão local é a opção mais simples e segura.

Link da publicação OpenCode: <https://opncd.ai/share/NZxPtJu2>

Print consumo de token:



Análise da refatoração: Eu considero que valeu a pena porque o custo de tempo e token foi relativamente baixo e houve pouco “desvio de exploração”, então o saldo foi positivo.

3.2.4 Ocorrência Número 4

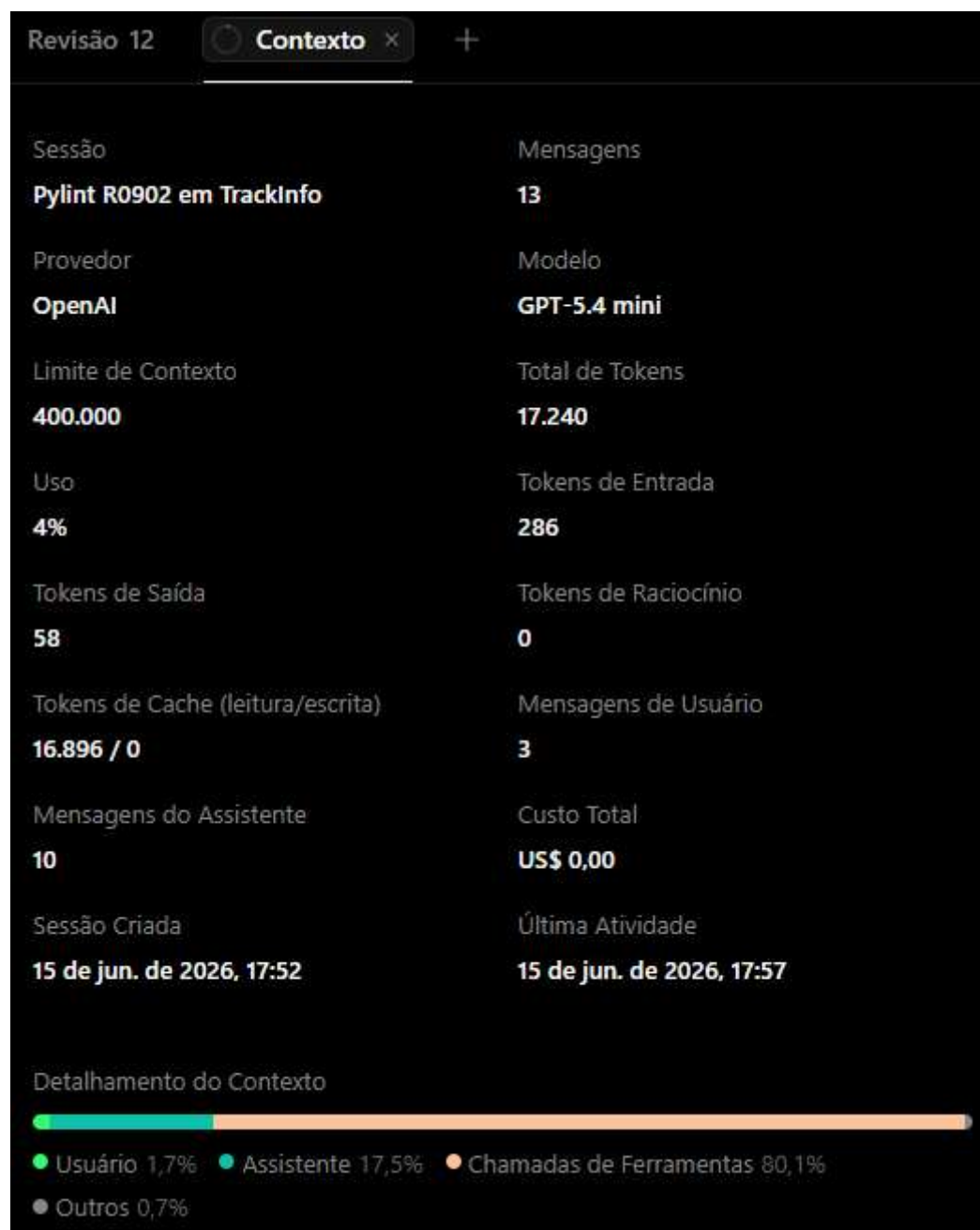
Tabela de identificação

module	obj	message
beets.autotag.hooks	TrackInfo	Too many instance attributes (24/7)

Seguiu o plano de refatoração: Sim. No primeiro planejamento já recebi duas propostas e escolhi a que se encaixa melhor, que é a de supressão local.

Link da publicação OpenCode: <https://opncd.ai/share/ay2Gdz6y>

Print consumo de token:



Análise da refatoração: Eu considero que valeu a pena. foi uma conversa curta com decisão consistente e sem tentativa de refatoração desnecessária. Sugeri de primeira o padrão correto, então acredito que o custo de tempo e tokens foi baixo em relação ao resultado.

3.2.5 Ocorrência Número 5

Tabela de identificação

module	obj	message
beets.dbcore.db	Results	Too many instance attributes (9/7)

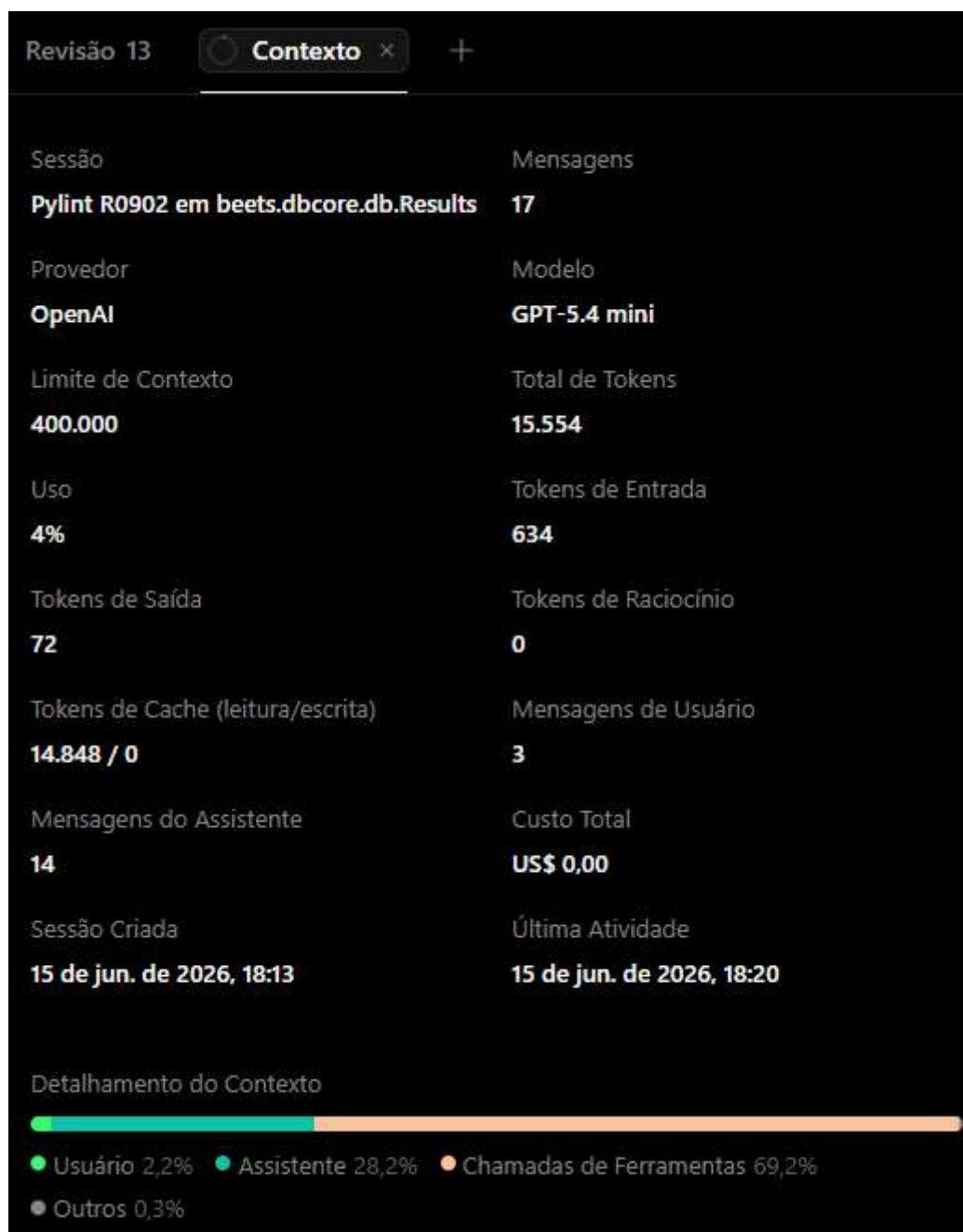
Seguiu o plano de refatoração:

Primeira sugestão: Não aprovei porque, novamente, foi sugerido fazer uma supressão estrutural, mesmo não sendo um problema de mau design.

Segunda sugestão: Segui o plano, pois o Result precisa desse estado interno por causa do cache e da materialização, então o R0902 aqui não indica um problema real de design. A supressão local é a solução mais simples e segura, sem mexer na estrutura ou na API.

Link da publicação OpenCode: <https://opncd.ai/share/Jz21Z6JB>

Print consumo de token:



Análise da refatoração: Considero que valeu a pena. Houve pouca divergência e a solução foi encontrada rápida.

3.2.6 Ocorrência Número 6

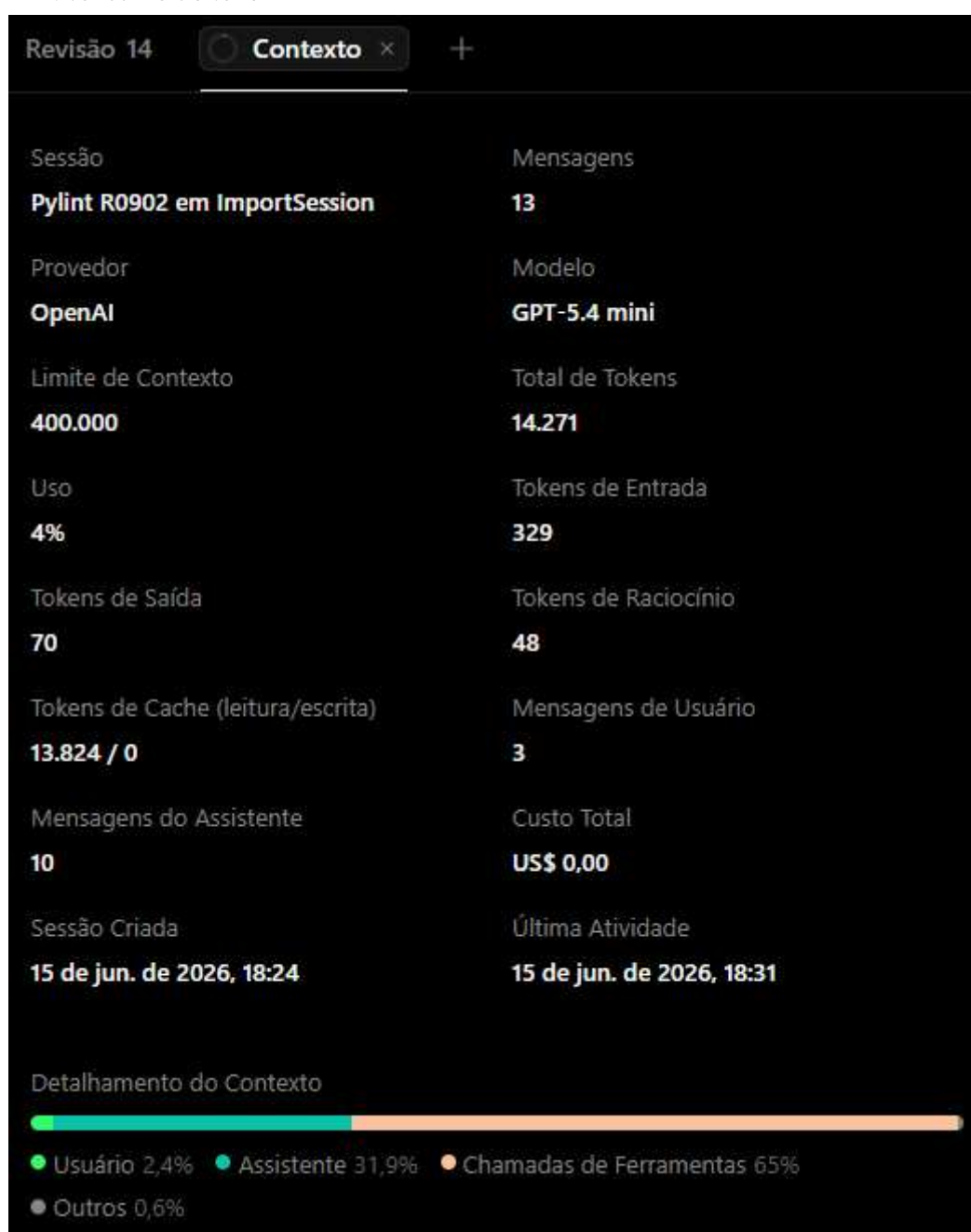
Tabela de identificação

module	obj	message
beets.importer.session	ImportSession	Too many instance attributes (10/7)

Seguiu o plano de refatoração: Eu não aprovei a primeira sugestão de plano porque caiu no mesmo problema de apenas satisfazer o lint. Novamente inseri o prompt: “**Evite mudanças estruturais só para satisfazer o lint. Se o warning não apontar um problema real de design, prefira uma supressão local em vez de alterar a implementação**”. Após isso, consegui guiá-lo melhor e aceitei o plano seguinte, que resolveu o problema, de fato.

Link da publicação OpenCode: <https://opncd.ai/share/J8d4sf2w>

Print consumo de token:



Análise da refatoração: Considero que valeu a pena, que o gasto de tempo e tokens foi proporcional ao problema ao problema.

3.2.7 Ocorrência Número 7

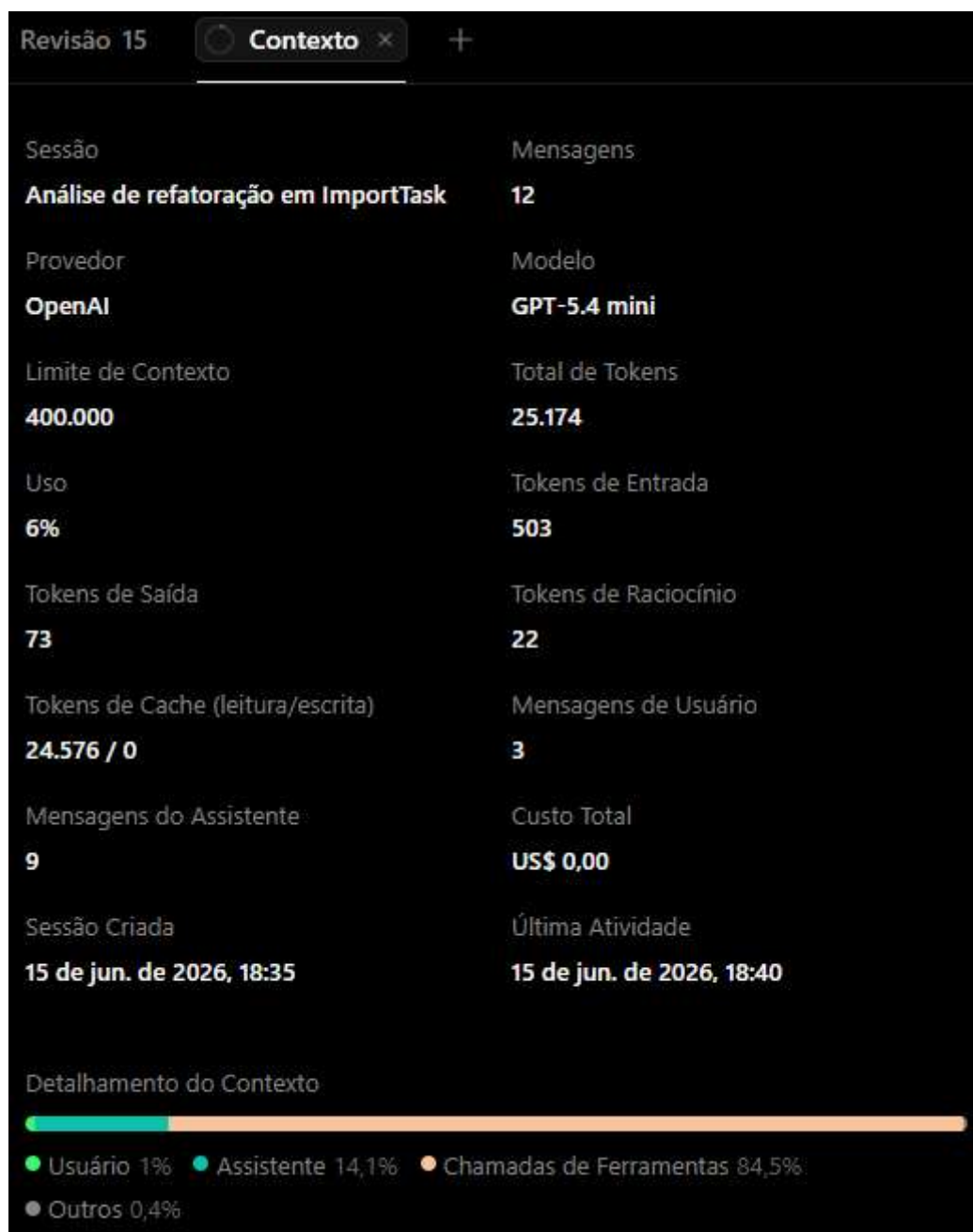
Tabela de identificação

module	obj	message
beets.importer.tasks	ImportTask	Too many instance attributes (13/7)

Seguiu o plano de refatoração: Novamente, não aprovei o primeiro plano por ser uma refatoração guiada por lint, não por design. Inserir um novo prompt novamente(o mesmo anterior) e somente assim ele conseguiu focar no problema, de fato. Após isso, aceitei a próxima sugestão, de supressão local, pois é a solução mais simples, segura e proporcional, sem introduzir complexidade desnecessária.

Link da publicação OpenCode: <https://opncd.ai/share/PGpYIk51>

Print consumo de token:



Análise da refatoração: Eu considero que não valeu muito a pena porque o gasto de token foi muito alto para uma conclusão simples: supressão local de um falso positivo de design.

3.3 too-many-branches

Quantidade de ocorrências: 7

3.3.1 Ocorrência Número 1

Tabela de identificação

module	obj	message
beets.autotag.distance	distance	Too many branches (19/12)

Seguiu o plano de refatoração: Sim, é um bom refactor porque reduz a complexidade dividindo responsabilidades sem alterar o comportamento.

Link da publicação OpenCode: <https://opncd.ai/share/e6DyktTc>

Print consumo de token:



Análise da refatoração: Eu considero que não valeu muito a pena. em um problema relativamente pequeno foram consumidos mais de 41k de tokens. Foi útil em questão de segurança, mas o custo foi muito alto.

3.3.2 Ocorrência Número 2

Tabela de identificação

module	obj	message
beets.importer.tasks	albums_in_dir	Too many branches (17/12)

Seguiu o plano de refatoração: Sim. `albums_in_dir` mistura 3 responsabilidades reais, o que aumenta risco de bug e dificulta mudanças. A extração proposta reduz complexidade ciclomática sem mudar comportamento e melhora testabilidade.

Link da publicação OpenCode: <https://opncd.ai/share/w0gvqKXF>

Print consumo de token:



Análise da refatoração: Considero que não valeu muito a pena, porque houve um custo alto de tokens. quase 24k, considerando o R0912.

3.3.3 Ocorrência Número 3

Tabela de identificação

module	obj	message
--------	-----	---------

beets.library.models	Item.destination	Too many branches (14/12)
----------------------	------------------	---------------------------

Seguiu o plano de refatoração: Sim. O método mistura várias responsabilidades (seleção de formato, normalização, asciify, legalização e decisão de retorno), o que dificulta manutenção e testes. A extração proposta reduz complexidade sem alterar comportamento e melhora legibilidade.

Link da publicação OpenCode: <https://opncd.ai/share/5bVKX1el>

Print consumo de token:



Análise da refatoração: Eu acho que não valeu muito a pena porque o custo em tokens para analisar/refatorar só um R0912 foi muito alto. Mesmo tendo sido rápido, talvez não compense muito.

3.3.4 Ocorrência Número 4

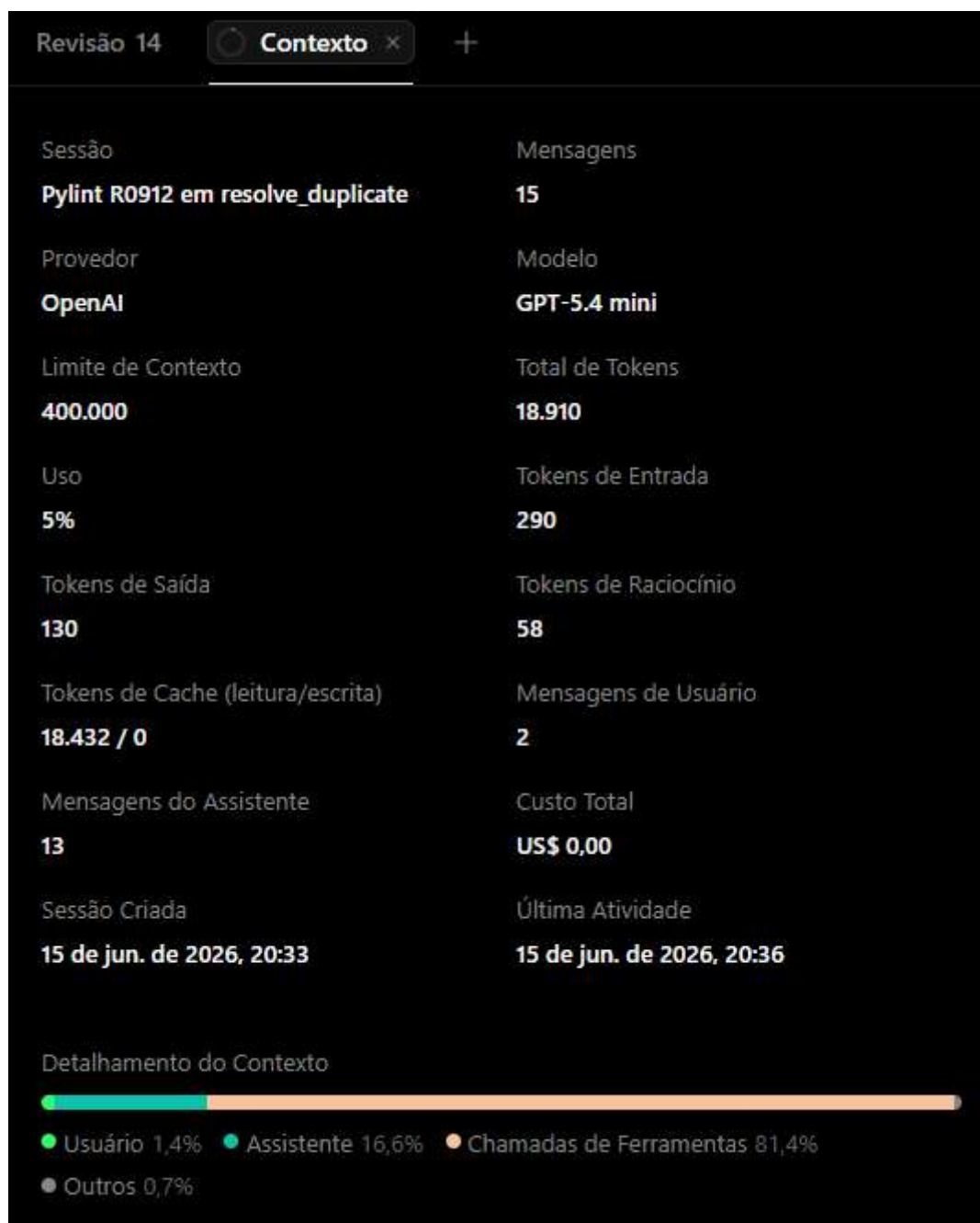
Tabela de identificação

module	obj	message
beets.ui.commands.import_session	TerminalImportSession.resolve_duplicate	Too many branches (14/12)

Seguiu o plano de refatoração: Sim, porque resolve_duplicate mistura I/O (prompt), lógica de decisão e execução de ações, o que aumenta complexidade e dificulta teste. A extração em dois helpers reduz branches reais do método principal e melhora legibilidade sem mudar comportamento.

Link da publicação OpenCode: <https://opncd.ai/share/BC9IFvGL>

Print consumo de token:



Análise da refatoração: A refatoração foi rápida em questão de tempo, mas eu acho que não valeu a pena considerando o alto consumo de tokens(quase 19k).

3.3.5 Ocorrência Número 5

Tabela de identificação

module	obj	message
--------	-----	---------

beets.util.bluelet	_event_select	Too many branches (18/12)
--------------------	---------------	---------------------------

Seguiu o plano de refatoração: Sim. O `_event_select` claramente mistura coleta, espera e reconstrução de resultado, o que gera muitas ramificações e baixa legibilidade. A divisão proposta separa bem as responsabilidades sem mudar comportamento.

Link da publicação OpenCode: <https://opncd.ai/share/1LsMtInb>

Print consumo de token:



Análise da refatoração: Eu considero que valeu a pena, já que além de rápida, o custo de tokens foi de menos de 16k. É um custo aceitável para a refatoração feita e ganho de legibilidade/manutenção.

3.3.6 Ocorrência Número 6

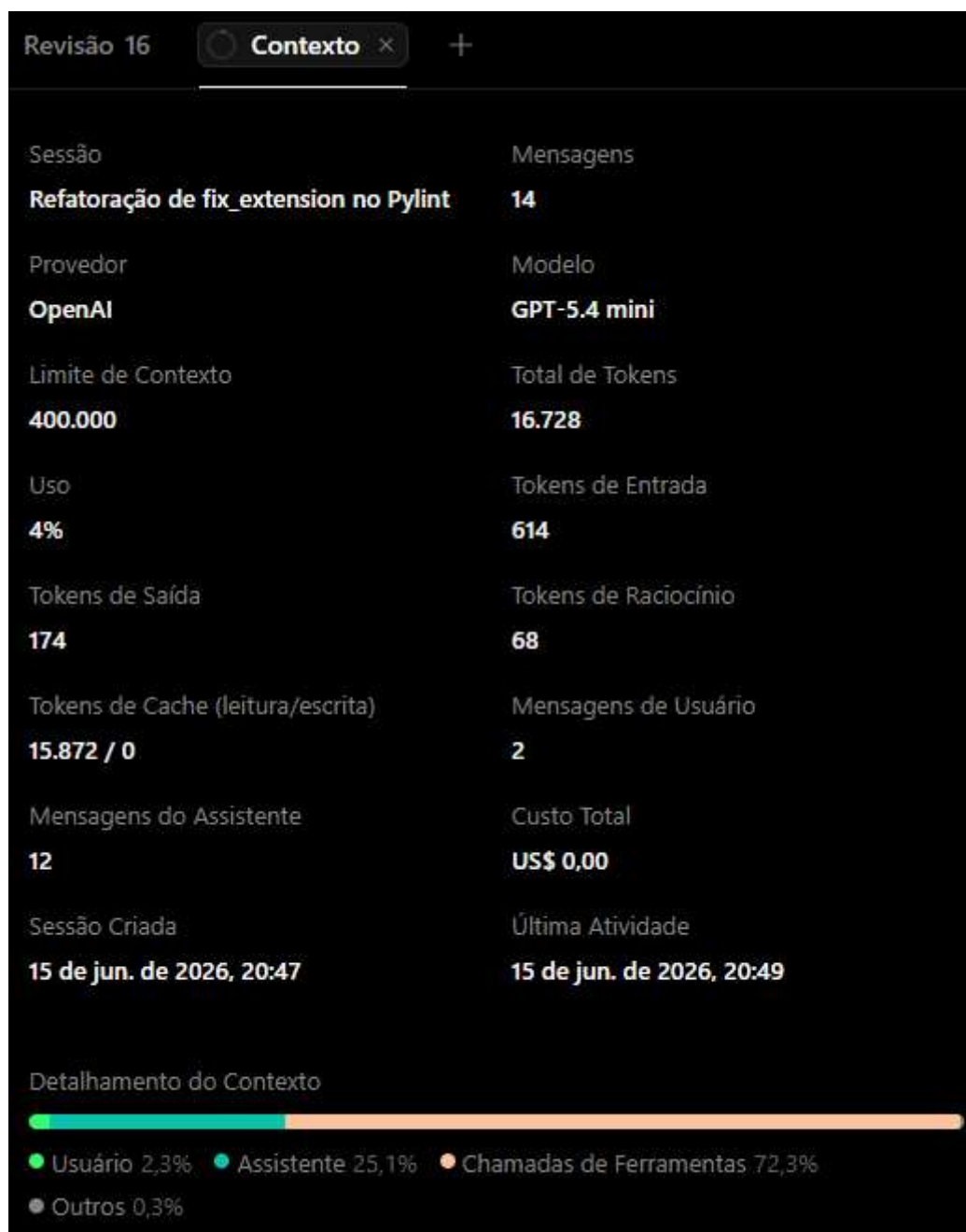
Tabela de identificação

module	obj	message
beets.util.extension	fix_extension	Too many branches (14/12)

Seguiu o plano de refatoração: Sim. O fix_extension concentra várias responsabilidades, o que aumenta branches e dificulta manutenção. A divisão sugerida melhora a legibilidade e testabilidade sem alterar o comportamento.

Link da publicação OpenCode: <https://opncd.ai/share/e1AyO0ho>

Print consumo de token:



Análise da refatoração: Eu considero que 16.7k de tokens é um custo aceitável para esse tipo de refatoração, e o uso de ferramentas não explodiu. Houve uma separação clara de responsabilidades e redução real de complexidade.

3.3.7 Ocorrência Número 7

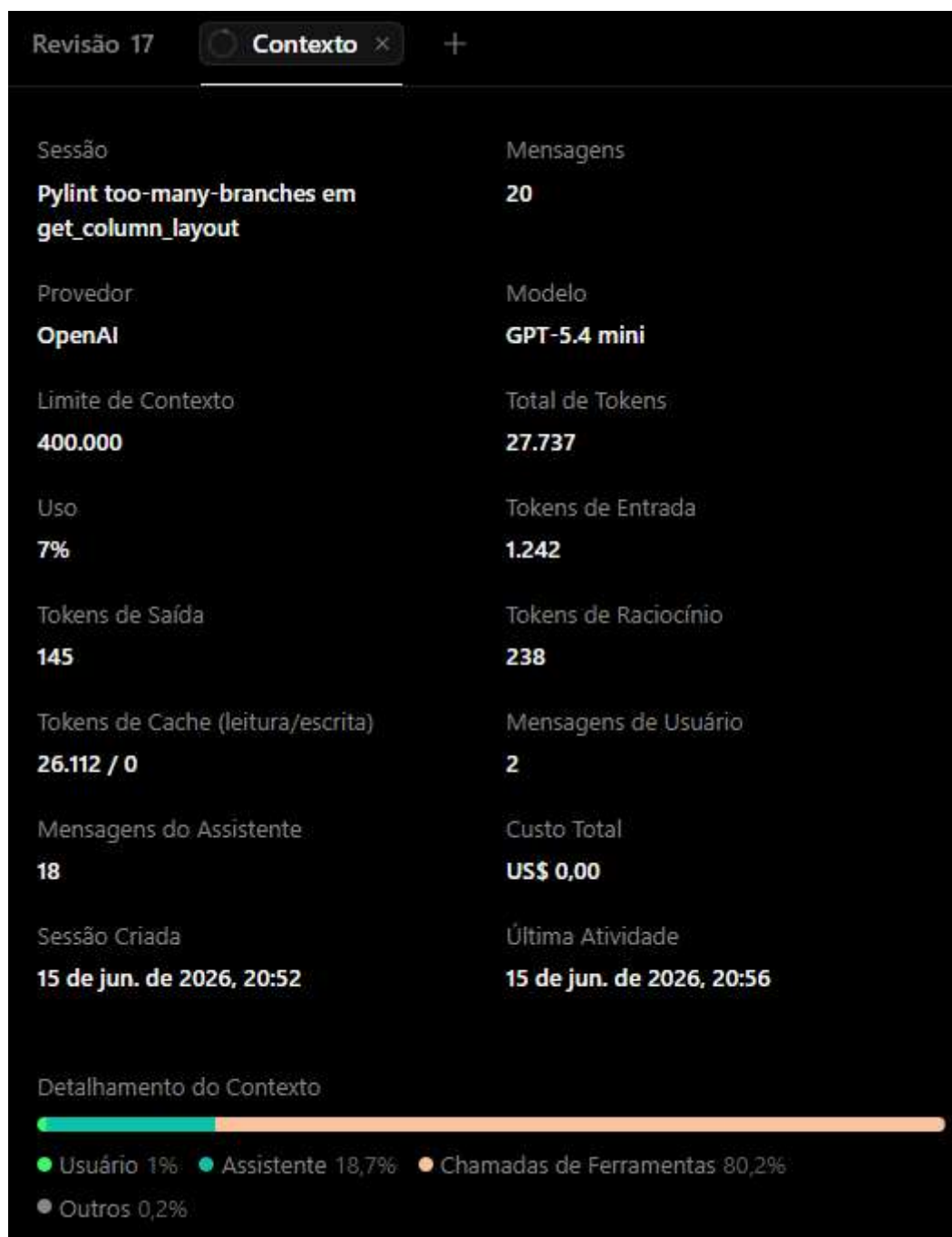
Tabela de identificação

module	obj	message
beets.util.layout	get_column_layout	Too many branches (17/12)

Seguiu o plano de refatoração: Sim. A `get_column_layout` concentra muita lógica de formatação e decisão em um único fluxo, gerando muitos branches reais. A extração proposta melhora legibilidade e testabilidade sem alterar comportamento, e mantém a função principal como orquestradora.

Link da publicação OpenCode: <https://opncd.ai/share/mkaQVLmz>

Print consumo de token:



Análise da refatoração: Eu considero que não valeu a pena. Foram usados mais de 27k de tokens para resolver um único R0912, mesmo com uma boa refatoração. O custo de coordenação e ferramentas foi desproporcional ao problema.

4. Métricas Após a Refatoração

4.1 Radon Após a Refatoração

4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
beets/util/hidden.py	1	9	9	9	B	is_hidden
beets/util/config.py	3	8.33	17	25	C	sanitize_pairs
beets/ui/commands/utls.py	1	8	8	8	B	do_query
beets/ui/commands/update.py	5	8	14	40	C	_resolve_update_fields
beets/ui/commands/config.py	2	8	10	16	B	config_func

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
beets/context.py	3	1	1	3	A	get_music_dir
beets/library/exceptions.py	4	1	1	4	A	__init__
beets/library/__init__.py	1	1	1	1	A	__getattr__
beets/ui/commands/__init__.py	1	1	1	1	A	__getattr__
beets/test/_common.py	16	1.19	2	19	A	item

4.1.2 Complexidade Ciclomática por Função Após a Refatoração

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
beets/ui/commands/import_/session.py	function	choose_candidate	C	15
beets/util/functemplate.py	method	parse_expression	C	15
beets/ui/commands/modify.py	function	modify_items	C	16
beets/util/config.py	function	sanitize_pairs	C	17

beets/ui/commands/move.py	function	move_items	D	22
---------------------------	----------	------------	---	----

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
beets/context.py	function	get_music_dir	A	1
beets/context.py	function	set_music_dir	A	1
beets/context.py	function	music_dir	A	1
beets/logging.py	function	getLogger	A	1
beets/logging.py	function	getLogger	A	1

4.1.3 Índice de Manutenibilidade Após a Refatoração

Tabela de piores métricas

arquivo	mi	rank_mi
beets/library/models.py	6.1095	C
beets/dbcore/db.py	12.5204	B
beets/importer/tasks.py	13.123	B
beets/dbcore/query.py	24.2219	A
beets/util/__init__.py	26.3283	A

Tabela de melhores métricas

arquivo	mi	rank_mi
beets/importer/__init__.py	100.0	A
beets/context.py	100.0	A
beets/exceptions.py	100.0	A
beets/ui/commands/__init__.py	100.0	A
beets/ui/commands/version.py	100.0	A

4.1.4 Métricas Brutas Após a Refatoração

loc total	lloc total	sloc total	comments total	multi
21217	10575	12568	1878	2739

4.2 CodeCarbon Após a Refatoração

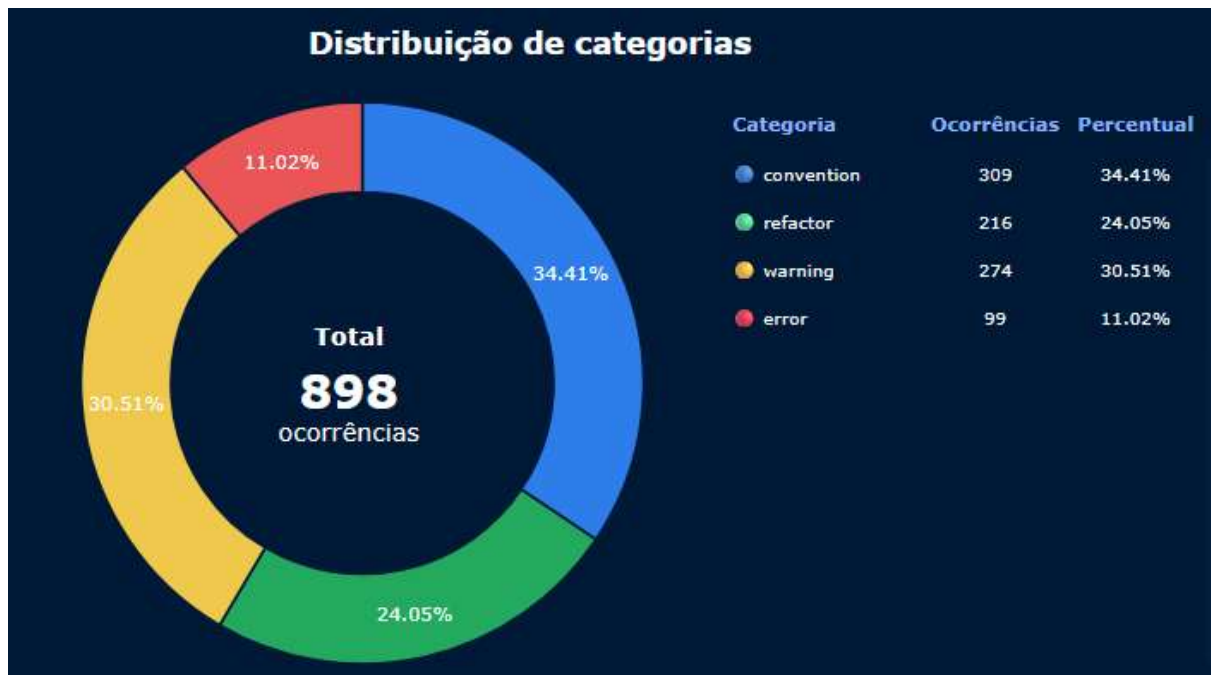
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
1.6269414	3.53E-07	2.17E-07	4.69E-07	3.13E-06	3.59E-06

4.3 Pylint Após a Refatoração

4.3.1 Distribuição do Smells Após a Refatoração



4.3.2 Distribuição dos Problemas de Código Após a Refatoração



4.3.3 Arquivos mais Críticos Após a Refatoração



4.3.4 Score Projeto Após a Refatoração



4.4 Pytest Após a Refatoração

4.4.1 Cobertura de testes Após a Refatoração

Code Coverage Report					
File	statements	missing	excluded	coverage	
beetsl_init_.py	18	5	0		70%
beetsl_main_.py	4	4	0		0%
beetslautotag_init_.py	12	4	0		57%
beetslautotagdistance.py	278	31	5		86%
beetslautotaghooks.py	217	1	2		99%
beetslautotagmatch.py	213	16	9		90%
beetscontext.py	13	0	0		100%
beetsdbcorel_init_.py	5	0	0		100%
beetsdbcoreldb.py	626	25	8		94%
beetsdbcorelpathutils.py	29	1	0		95%
beetsdbcorelquery.py	427	29	7		92%
beetsdbcorelqueryparse.py	79	1	4		98%
beetsdbcorelsort.py	107	8	2		94%
beetsdbcoreltypes.py	237	5	8		97%
beetsexceptions.py	1	0	0		100%
beetsimporter_init_.py	3	0	0		100%
beetsimporterlsession.py	142	11	9		91%
beetsimporterlstates.py	169	13	4		89%
beetsimporterlstate.py	57	5	2		89%
beetsimporterltasks.py	558	28	6		93%
beetslibrary_init_.py	9	0	0		100%
beetslibraryexceptions.py	14	0	0		100%
beetslibraryfields.py	2	0	0		100%
beetslibrarylibrary.py	88	5	4		94%
beetslibrarymigrations.py	114	1	3		97%

beetslibrary\models.py	652	60	6		88%
beetslibrary\queries.py	22	0	0		100%
beets\logging.py	58	1	12		96%
beets\mediafile.py	7	7	0		0%
beets\metadata_plugins.py	164	23	9		82%
beets\plugins.py	242	19	17		89%
beets\ui__init__.py	426	53	2		86%
beets\ui\commands__init__.py	18	0	0		100%
beets\ui\commands\completion.py	52	41	0		15%
beets\ui\commands\config.py	43	6	0		83%
beets\ui\commands\fields.py	20	1	0		91%
beets\ui\commands\help.py	13	0	0		100%
beets\ui\commands\import_init__.py	107	57	0		40%
beets\ui\commands\import_display.py	201	27	5		82%
beets\ui\commands\import_session.py	238	75	0		64%
beets\ui\commands\list.py	13	0	0		100%
beets\ui\commands\modify.py	67	3	0		96%
beets\ui\commands\move.py	74	15	2		75%
beets\ui\commands\remove.py	36	0	0		100%
beets\ui\commands\stats.py	35	5	0		81%
beets\ui\commands\update.py	94	13	0		84%
beets\ui\commands\utils.py	15	0	0		100%
beets\ui\commands\version.py	13	1	0		87%
beets\ui\commands\write.py	25	5	0		81%
beets\util__init__.py	496	94	5		80%
beets\util\airresizer.py	358	150	5		54%
beets\util\bluelet.py	351	259	0		20%
beets\util\color.py	56	0	0		100%
beets\util\config.py	42	0	2		100%
beets\util\deprecation.py	34	2	2		93%
beets\util\diff.py	47	0	4		100%
beets\util\extension.py	71	34	0		40%
beets\util\func_template.py	272	24	0		90%
beets\util\hidden.py	21	8	0		49%
beets\util\id_extractors.py	14	3	0		81%
beets\util\layout.py	157	18	3		85%
beets\util\lyrics.py	91	2	2		97%
beets\util\m3u.py	40	6	0		84%
beets\util\pathformats.py	7	0	4		100%
beets\util\pipeline.py	234	11	2		94%
beets\util\units.py	30	0	0		100%
beetsplug_utils__init__.py	2	0	0		100%
beetsplug_utils\art.py	94	30	0		65%
beetsplug_utils\musicbrainz.py	340	7	5		97%
beetsplug_utils\playcount.py	48	1	5		97%
beetsplug_utils\requests.py	90	0	6		98%
beetsplug_utils\svfs.py	21	0	2		96%
beetsplug\acousticbrainz.py	95	55	0		38%
beetsplug\advancedrewrite.py	74	7	0		88%
beetsplug\albumtypes.py	26	2	2		91%
beetsplug\badfiles.py	135	94	0		26%
beetsplug\bareasc.py	40	8	0		74%
beetsplug\beatport.py	239	139	15		39%
beetsplug\bpd_init__.py	884	862	4		2%
beetsplug\bpd\gstplayer.py	156	149	0		3%
beetsplug\bucket.py	117	0	0		100%

beetsplugchroma.py	267	125	5		49%
beetsplugconvert.py	337	73	5		74%
beetsplugdiscogs_init_.py	324	110	5		65%
beetsplugdiscogsstates.py	121	3	5		96%
beetsplugdiscogstypes.py	42	0	2		100%
beetsplugduplicates.py	165	67	0		54%
beetsplugedit.py	168	16	0		88%
beetsplugembedart.py	125	27	0		77%
beetsplugembedupdate.py	75	30	0		54%
beetsplugexport.py	103	10	2		88%
beetsplugfetchart.py	733	157	5		77%
beetsplugfilefilter.py	34	3	0		85%
beetsplugfromfilename.py	68	2	0		95%
beetsplugftintitle.py	113	9	3		89%
beetsplugfuzzy.py	26	0	0		97%
beetsplughook.py	38	5	0		84%
beetsplughate.py	36	19	0		44%
beetsplugimportadded.py	78	1	0		95%
beetsplugimportfeeds.py	79	11	0		80%
beetsplugimportsource.py	68	14	0		75%
beetspluginfo.py	121	17	0		84%
beetspluginline.py	68	9	0		86%
beetsplugipfs.py	206	160	0		20%
beetsplugkeyfinder.py	46	6	0		86%
beetspluglastgenre_init_.py	262	31	11		84%
beetspluglastgenreclient.py	41	18	7		51%
beetspluglastgenreutils.py	15	0	5		100%
beetspluglimit.py	43	4	0		86%
beetspluglistenbrainz.py	229	85	3		62%
beetspluglyrics.py	516	93	11		78%
beetsplugmbcollection.py	103	4	6		93%
beetsplugmbpseudo.py	156	15	5		86%
beetsplugmbsubmit.py	38	17	0		52%
beetsplugmbsync.py	79	8	0		84%
beetsplugmetasync_init_.py	69	10	2		85%
beetsplugmetasynclamark.py	39	20	0		39%
beetsplugmetasyncitunes.py	56	9	0		80%
beetsplugmissing.py	88	17	3		75%
beetsplugmpdstats.py	192	78	0		54%
beetsplugmusicbrainz.py	320	9	5		96%
beetsplugparentwork.py	92	15	2		78%
beetsplugpermissions.py	53	31	0		33%
beetsplugplay.py	112	14	0		85%
beetsplugplaylist.py	109	8	3		91%
beetsplugplexupdate.py	47	11	0		74%
beetsplugrandom.py	49	4	4		91%
beetsplugreplace.py	73	33	2		54%
beetsplugreplaygain.py	676	539	9		16%
beetsplugrewrite.py	50	1	0		98%
beetsplugscrub.py	67	24	0		66%
beetsplugsmartplaylist.py	216	15	3		91%
beetsplugspotify.py	395	109	6		69%
beetsplugsubsonicupdate.py	63	13	0		75%
beetsplugsubstitute.py	13	1	0		88%
beetsplugthe.py	53	6	0		82%
beetsplugthumbnails.py	145	34	1		77%

beetsplugtidal_init.py	185	18	35		88%
beetsplugtidalapi.py	83	56	4		26%
beetsplugtidal/session.py	50	30	2		34%
beetsplugtidalcase.py	128	0	4		99%
beetsplugtypes.py	25	1	0		95%
beetsplugwebb_init.py	272	73	0		71%
beetsplugzero.py	90	4	0		95%
extrarelease.py	113	101	0		9%
extract_metrics_after_codecarbon.py	28	28	0		0%
extract_metrics_after_pylintr.py	58	58	0		0%
extract_metrics_after_pytest.py	32	32	0		0%
extract_metrics_after_radon.py	111	111	0		0%
extract_score_after_pylintr.py	13	13	0		0%
Total	19827	5245	359		71%
Total	39654	10490	718		74%

4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração

Quantidade de testes que passaram: 2341

Quantidade de testes que falharam: 10

5. Comparação dos Resultados

A refatoração não comprometeu o funcionamento do sistema, pois todos os resultados dos testes permaneceram iguais. Além disso, houve um pequeno ganho em documentação e a simplificação de algumas funções.

Entretanto, o principal objetivo de melhorar a qualidade interna do código não foi plenamente alcançado, já que:

- o número de code smells aumentou;
- o tamanho do código cresceu;
- o índice geral de qualidade permaneceu inalterado.

Assim, considero que a refatoração foi segura do ponto de vista funcional, mas teve baixo impacto positivo na qualidade estrutural do projeto, sendo necessárias novas intervenções para reduzir os code smells e melhorar efetivamente a manutenibilidade do sistema.

6. Percepções sobre a Prática

Experiência geral com a atividade: Eu considero que essa é uma prática muito metódica. Penso que, mesmo que o conceito seja simples, exige uma quantidade boa de tempo e atenção.

Code smells:

Too-Many-Statements: São métodos muito longos que dificultam a leitura, manutenção e identificação de erros. A refatoração que fiz torna o código mais modular e compreensível.

Too-Many-Instance-Attributes: São classes com muitos atributos que tendem a concentrar responsabilidades excessivas. Reduzir eles melhora a coesão e facilita a manutenção.

Too-Many-Branches: São muitos desvios condicionais que aumentam a complexidade do código e dificultam os testes. A refatoração que fiz reduz a complexidade ciclomática e melhora a legibilidade.

Eu considero que também é muito importante refatorar os **Duplicated Code**, porque alterações precisam ser feitas em vários locais ao mesmo tempo, aumentando o risco de erros e tornando a manutenção mais difícil e custosa.

Benefícios observados

A refatoração preservou o comportamento do sistema, mantendo os mesmos resultados nos testes. Além disso, contribuiu para uma melhor organização e modularização do código, acompanhada de um pequeno aumento na documentação interna.

Não-benefícios observados

Apesar das melhorias estruturais, o número de code smells aumentou (de 855 para 893), o Quality Score permaneceu em 8,54/10 e não houve ganhos significativos nas métricas de manutenibilidade. Também observei um leve aumento no tamanho do código.

Uso da LLM na refatoração: No geral, ela compreendeu bem os problemas. Porém, em alguns casos ela ficava alucinando, tentando apenas satisfazer o lint, mas não resolver efetivamente o problema. Eu acho que ela acelerou o processo e não complicou tanto. Eu acho que os projetos Open Source podem se beneficiar bastante, mas no beets, por exemplo, diz que a IA não deve ser um fator contribuinte na contribuição. Eu também usei uma LLM para me ajudar a projetar códigos de geração de gráficos mais rápidos, pois ela simplesmente NÃO CONSEGUIU gerar um gráfico como imagem da forma correta.

Contribuição com projeto open source: A maior dificuldade foi descobrir a forma certa de contribuir no projeto. O README muito grande, cheio de regras e requisitos. O meu PR, por exemplo, não passou porque era o meu primeiro, e por ser o primeiro, não poderia ser algo tão complexo, deveria ser algo mais simples.

7. Links

- Link do fork: <https://github.com/kendriks/beets.git>
- Link do PR 1: <https://github.com/beetbox/beets/pull/6746>
- Link do COMMIT 2:
- <https://github.com/kendriks/beets/commit/20914755dd8870ed827dbe31dcb20742a5af5264>
- Link do COMMIT 3:
- <https://github.com/kendriks/beets/commit/4f6e5db07d1d24110bfd3b058f991d9619df4034>
- Link do COMMIT de Scripts e Métricas:
- <https://github.com/kendriks/beets/commit/67f6344b4e2c51429929a775d31c69b416303343>

- Link do COLAB para melhor visualização dos gráficos:
https://colab.research.google.com/drive/1MnRLe89E7_po6VFbIlyd9cNjWG_pJ3Xa?usp=sharing